Unit Of Work (UOW)

Overview



What is Unit Of Work?

Benefits of UOW

How is UOW different from a service?

What is Unit Of Work?

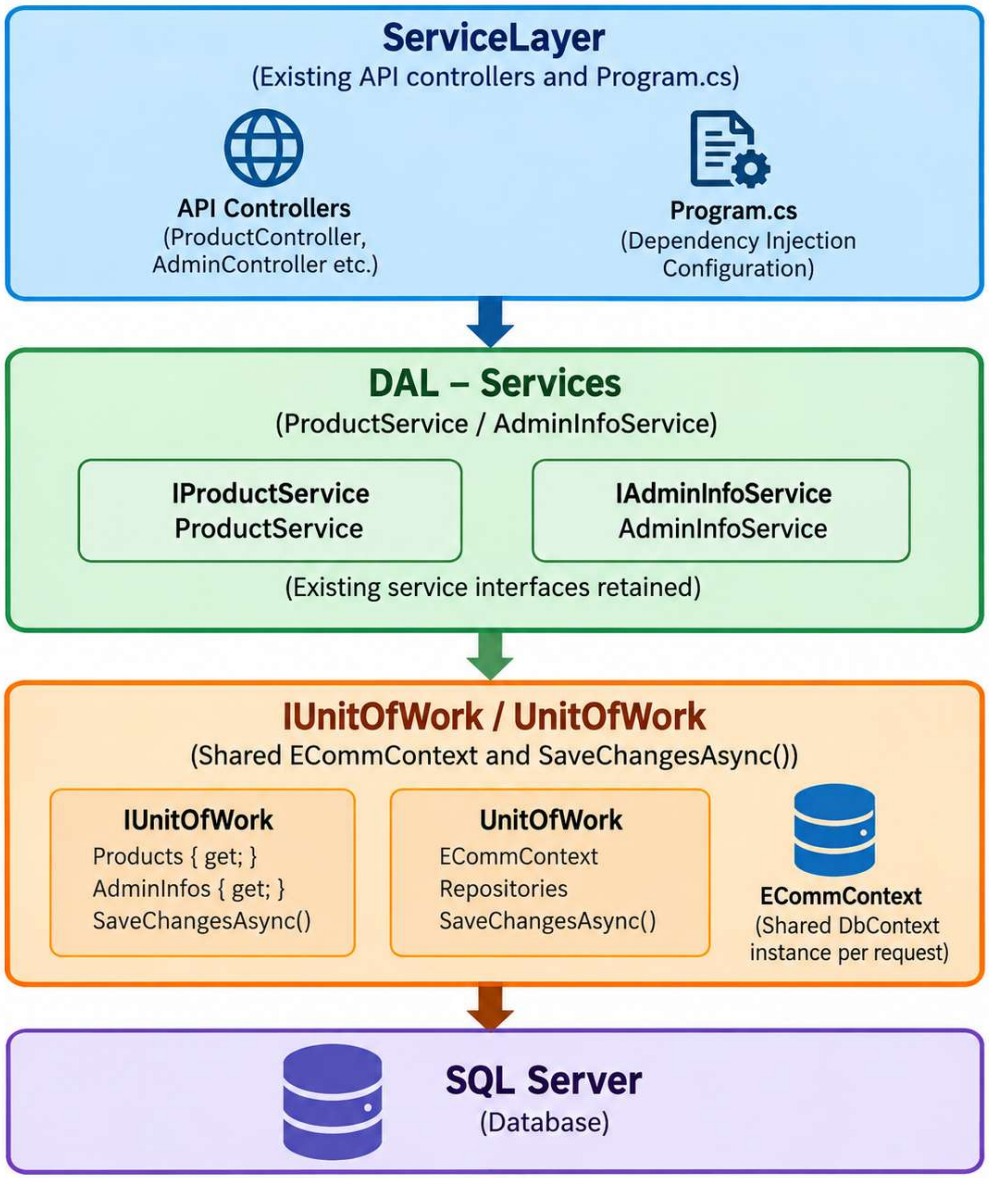
- The Unit of Work (UOW) pattern is a design pattern that coordinates database operations performed by different services and provides a common place to save changes to the database.

In simple words: Unit of Work manages database changes through a shared DbContext and commits those changes using a single SaveChangesAsync() method.

In your demo, we have two services:

- **ProductService** — handles product operations.
- **AdminInfoService** — handles admin information and validation.

Both services can use the same ECommContext through the UnitOfWork class.



The workflow is:

- A request comes to an API controller in ServiceLayer.
- The controller calls the appropriate service in DAL.
- The service uses IUnitOfWork to access ECommContext.
- For write operations, the service calls SaveChangesAsync() through UOW.
- Entity Framework Core sends the pending database changes to SQL Server.

Why do we need UOW?

1. Centralized database saving

Services use the same SaveChangesAsync() method provided by UOW instead of directly managing the save operation.

2. Shared DbContext

Services participating in the same unit of work can use one ECommContext instance and its change tracker.

3. Better separation of responsibilities

ProductService handles product logic, AdminInfoService handles admin logic, and UOW coordinates database access and saving.

4. Easier testing and maintenance

Services depend on the IUnitOfWork interface, making it easier to substitute a test implementation and change the database coordination code.

Problem 1: Too much responsibility in one place

Controllers may contain validation, business rules, and database operations. Changes become difficult to manage.

Problem 2: Tight coupling to the database

Business logic that directly depends on Entity Framework Core can be harder to test or reuse with another data-access implementation.

Problem 3: Changes affect multiple parts

Changing how data is stored might require changes to business logic and API code if responsibilities are mixed.

How is UOW different from a service?

Component	Responsibility in our Example
ProductService	Add, update, search and delete products
AdminInfoService	Validate administrator credentials
IUnitOfWork	Defines the common database coordination contract
UnitOfWork	Uses ECommContext and exposes SaveChangesAsync()
ECommContext	Represents the EF Core database session and tracks entity changes
SQL Server	Stores persistent data